

GUÍA PRÁCTICA DE LABORATORIO

API REST con Node.js 20, Express 5 y MySQL: CRUD de usuarios seguro, validado y probado

Datos generales	
Curso	JavaScript Avanzado
Tema	Desarrollo de una API REST por capas con Node.js, Express y MySQL
Modalidad	Laboratorio (práctica guiada en clase)
Duración estimada	180 minutos
Docente	Ing. David Reyna Barreto
Estudiante	
Sección / Fecha	

1. Logro de aprendizaje

Al finalizar la práctica, **implementas una API REST** con Node.js 20 y Express 5 que realiza las operaciones CRUD (crear, leer, actualizar y eliminar) sobre una base de datos MySQL, **organizada por capas** (rutas, controladores, modelos y middlewares), **validando** los datos de entrada, **maneja los errores** de forma centralizada y **verificando su funcionamiento** con pruebas manuales y automatizadas.

Lo que vas a construir

Una API para gestionar usuarios con cinco endpoints. No es un ejercicio de juguete: aplicarás las mismas prácticas que se exigen en un proyecto profesional, como proteger las credenciales, impedir que el cliente modifique columnas que no debe, responder con los códigos HTTP correctos y no exponer detalles internos cuando algo falla.

Método	Ruta	Acción	Respuestas posibles
GET	/api/users	Listar todos los usuarios	200
GET	/api/users/:id	Obtener un usuario	200, 400, 404
POST	/api/users	Crear un usuario	201, 400, 409
PUT	/api/users/:id	Actualizar un usuario	200, 400, 404, 409
DELETE	/api/users/:id	Eliminar un usuario	204, 404

2. Requisitos previos

2.1 Conocimientos

- JavaScript moderno: funciones flecha, desestructuración, módulos CommonJS (`require` / `module.exports`).
- Promesas y `async/await`.
- Conceptos básicos de HTTP (métodos, cabeceras, cuerpo JSON) y de SQL (`SELECT`, `INSERT`, `UPDATE`, `DELETE`).

2.2 Software

Herramienta	Versión	Para qué se usa
Node.js	20.12 o superior	Entorno de ejecución de JavaScript en el servidor
MySQL o MariaDB	MySQL 8 / MariaDB 10.6+ (XAMPP sirve)	Base de datos relacional
Visual Studio Code	Actual	Editor de código
Postman o Thunder Client	Actual	Probar los endpoints manualmente
Git	2.x	Control de versiones y entrega del proyecto

 **Nota:** Si tienes otra versión de Node instalada, usa **nvm-windows** (o **nvm** en macOS/Linux) para cambiar: `nvm install 20` y luego `nvm use 20`.

3. Fundamentos

3.1 Arquitectura por capas

Separar el código en capas permite que cada archivo tenga una sola responsabilidad. Si mañana cambias MySQL por PostgreSQL, solo tocas el modelo; si cambian las reglas de validación, solo tocas el middleware. Una petición recorre las capas en este orden:

Capa	Archivo(s)	Responsabilidad
1. Aplicación	app.js, index.js	Configura Express (CORS, JSON), monta las rutas y arranca el servidor
2. Rutas	routes/users.js	Asocia cada método y URL con su cadena de middlewares y su controlador
3. Middlewares	middlewares/validation.js	Valida y filtra los datos antes de que lleguen al controlador
4. Controlador	controllers/userController.js	Decide qué responder (200, 201, 404...) usando el modelo
5. Modelo	models/userModel.js	Ejecuta las consultas SQL; no sabe nada de HTTP
6. Datos	db.js, config.js	Pool de conexiones y lectura de variables de entorno
Transversal	middlewares/errorHandler.js	Atrapa cualquier error y responde sin exponer detalles internos

3.2 Códigos de estado HTTP que usarás

Una API bien diseñada comunica el resultado con el código de estado, no solo con el cuerpo de la respuesta. El cliente (una web, una app móvil) toma decisiones en función de ese número (MDN Web Docs, s. f.).

Código	Significado	Cuándo lo devuelve esta API
200 OK	Éxito	Consulta o actualización correcta
201 Created	Recurso creado	POST exitoso; devuelve el usuario creado
204 No Content	Éxito sin cuerpo	DELETE exitoso
400 Bad Request	Petición inválida	Datos faltantes o mal formados, id no numérico, JSON roto
404 Not Found	No existe	El usuario o la ruta no existen
409 Conflict	Conflicto	El email ya está registrado

Código	Significado	Cuándo lo devuelve esta API
500 Internal Server Error	Error del servidor	Falla inesperada (por ejemplo, la base de datos no responde)

3.3 Conceptos clave

Pool de conexiones. En lugar de abrir una sola conexión a MySQL (que se pierde si el servidor la cierra), se mantiene un grupo de conexiones reutilizables. Si una falla, el pool abre otra automáticamente (MySQL2, s. f.).

Consultas parametrizadas. Los valores del usuario nunca se concatenan dentro del SQL; se envían por separado con el marcador `?`. Así se evita la inyección SQL.

Asignación masiva (mass assignment). Ocurre cuando el servidor copia todo el cuerpo de la petición a la base de datos. Un atacante podría enviar `"role": "admin"` y otorgarse privilegios. La defensa recomendada es una **lista blanca** de campos permitidos (OWASP Foundation, s. f.).

Errores asíncronos en Express 5. Desde la versión 5, si una función `async` de una ruta lanza un error o una promesa es rechazada, Express llama automáticamente a `next(err)`, así que ya no necesitas un `try/catch` en cada controlador (Express.js, s. f.).

Variables de entorno. Las credenciales nunca se escriben en el código. Node 20.12 incorporó `process.loadEnvFile()`, que lee un archivo `.env` sin instalar librerías externas (Node.js, s. f.).

4. Desarrollo de la práctica

Sigue los pasos en orden. Al final de varios de ellos encontrarás un **punto de control**: no avances hasta cumplirlo.

Paso 1. Verificar el entorno

Abre una terminal (en VS Code: **Terminal › New Terminal**) y comprueba las versiones instaladas:

```
node -v      # debe mostrar v20.12.0 o superior
npm -v
git --version
```

Inicia MySQL (desde el panel de XAMPP o como servicio). Anota el **puerto** en el que corre: por defecto es **3306**; en algunos laboratorios se configura **3309**.

✓ **Punto de control:** `node -v` muestra una versión 20.12 o superior y MySQL está en ejecución.

Paso 2. Crear la base de datos

Crea un archivo `schema.sql` con el siguiente contenido y ejecútalo en phpMyAdmin (pestaña SQL), MySQL Workbench o desde la terminal con `mysql -u root -p < schema.sql`.

```
schema.sql
-- Crea la base de datos y la tabla que usa la API
CREATE DATABASE IF NOT EXISTS crud_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE crud_db;

CREATE TABLE IF NOT EXISTS users (
  id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Observa la restricción **UNIQUE** sobre `email`: impide dos usuarios con el mismo correo. Más adelante la API traducirá ese error de MySQL en una respuesta **409 Conflict**.

✓ **Punto de control:** En tu gestor de base de datos existe `crud_db` con la tabla `users` vacía.

Paso 3. Crear el proyecto e instalar dependencias

```
mkdir api-node-2025
cd api-node-2025
npm init -y
npm install express mysql2 cors
npm install -D supertest
npm pkg set scripts.start="node index.js" scripts.dev="node --watch index.js" scripts.test="node --test"
npm pkg set engines.node=">=20.12.0"
```

Crea también un archivo `.nvmrc` que contenga solo el número **20**, y la siguiente estructura de carpetas:

```
api-node-2025/
├── controllers/
│   └── userController.js
├── middlewares/
│   ├── errorHandler.js
│   └── validation.js
├── models/
│   └── userModel.js
├── routes/
│   └── users.js
├── test/
│   └── users.test.js
├── .env.example
├── .gitignore
├── .nvmrc
├── app.js
├── config.js
├── db.js
├── index.js
├── package.json
└── schema.sql
```

Paquete	Tipo	Función
express	dependencia	Framework HTTP: rutas, middlewares y respuestas
mysql2	dependencia	Cliente de MySQL con soporte de promesas y pool
cors	dependencia	Permite que un frontend en otro dominio consuma la API
supertest	desarrollo	Envía peticiones HTTP a la app dentro de los tests

 **Nota:** `body-parser` y `dotenv` ya no son necesarios: Express 5 incluye `express.json()` y Node 20.12 incluye `process.loadEnvFile()`.

Paso 4. Configurar las variables de entorno

Crea `.env.example` (se sube a GitHub como plantilla, sin contraseñas reales):

```
.env.example
# Copia este archivo como .env y completa tus datos
PORT=3000

DB_HOST=localhost
DB_PORT=3309
DB_USER=root
DB_PASSWORD=
DB_NAME=crud_db

# Orígenes permitidos para CORS, separados por comas (vacío = todos)
CORS_ORIGIN=http://localhost:5173
```

Cópialo como `.env` y coloca tus datos reales. En Windows: `copy .env.example .env`; en macOS/Linux: `cp .env.example .env`. Luego crea `.gitignore` para que `.env` y `node_modules` nunca se suban:

```
.gitignore
node_modules/
.env
*.log
```

Ahora crea `config.js`, que carga el `.env` con la función nativa de Node 20. Si el archivo no existe (por ejemplo, en un servidor donde las variables se configuran en el sistema), simplemente continúa:

config.js

```
// Carga las variables de .env con la función nativa de Node (>= 20.12), sin dotenv.
// Si no existe .env se usan las variables de entorno del sistema.
try {
  process.loadEnvFile();
} catch (err) {
  if (err.code !== 'ENOENT') throw err;
}
```

 **Seguridad:** Nunca subas el archivo `.env` al repositorio. Si por error lo haces, cambia la contraseña de la base de datos: borrar el archivo no la elimina del historial de Git.

Paso 5. Crear el pool de conexiones

db.js

```
require('./config');
const mysql = require('mysql2/promise');

// Pool de conexiones: se recupera solo si MySQL cierra una conexión
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  port: Number(process.env.DB_PORT) || 3306,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
  connectionLimit: Number(process.env.DB_POOL_SIZE) || 10,
});

module.exports = pool;
```

Fíjate en que se importa `mysql2/promise`: así cada consulta devuelve una promesa y puedes usar `await`. El pool no se conecta hasta la primera consulta.

Paso 6. Crear el modelo

El modelo concentra todo el SQL. Cada método es `async` y devuelve datos simples; no conoce `req` ni `res`.

models/userModel.js

```
const db = require('../db');

// Columnas que se devuelven al cliente (nunca SELECT *)
const PUBLIC_COLUMNS = 'id, name, email';

const User = {
  async getAll() {
    const [rows] = await db.query(`SELECT ${PUBLIC_COLUMNS} FROM users ORDER BY id`);
    return rows;
  },

  async getById(id) {
    const [rows] = await db.query(`SELECT ${PUBLIC_COLUMNS} FROM users WHERE id = ?`, [id]);
    return rows[0] || null;
  },

  async create({ name, email }) {
    const [result] = await db.query('INSERT INTO users (name, email) VALUES (?, ?)', [name, email]);
    return result.insertId;
  },

  // Solo actualiza los campos recibidos; devuelve true si el usuario existía
  async update(id, fields) {
    const [result] = await db.query('UPDATE users SET ? WHERE id = ?', [fields, id]);
```

```

    return result.affectedRows > 0;
  },

  async delete(id) {
    const [result] = await db.query('DELETE FROM users WHERE id = ?', [id]);
    return result.affectedRows > 0;
  },
};

module.exports = User;

```

Tres decisiones importantes: se usan **consultas parametrizadas (?)**; se devuelven solo las columnas públicas mediante **PUBLIC_COLUMNS** en lugar de **SELECT ***, para no filtrar datos sensibles si la tabla crece; y **update** y **delete** devuelven **affectedRows > 0**, lo que permite al controlador saber si el usuario existía.

Paso 7. Crear el middleware de validación

Este middleware se ejecuta **antes** del controlador. Filtra el cuerpo con una lista blanca, valida formatos y deja los datos limpios en **req.userData**.

```

middlewares/validation.js
// Campos que el cliente puede enviar. Cualquier otro (id, role, etc.) se ignora.
const ALLOWED_FIELDS = ['name', 'email'];
const EMAIL_REGEX = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

function pickAllowed(body = {}) {
  const data = {};
  for (const field of ALLOWED_FIELDS) {
    if (body[field] !== undefined) data[field] = typeof body[field] === 'string' ? body[field].trim() : body[field];
  }
  return data;
}

function checkFields(data) {
  const errors = [];
  if ('name' in data && (typeof data.name !== 'string' || data.name.length < 2 || data.name.length > 100)) {
    errors.push('name debe ser un texto de 2 a 100 caracteres');
  }
  if ('email' in data && (typeof data.email !== 'string' || !EMAIL_REGEX.test(data.email) || data.email.length > 150)) {
    errors.push('email no es válido');
  }
  return errors;
}

// POST: name y email obligatorios
exports.validateCreate = (req, res, next) => {
  const data = pickAllowed(req.body);
  const errors = checkFields(data);
  if (!data.email) errors.unshift('email es obligatorio');
  if (!data.name) errors.unshift('name es obligatorio');
  if (errors.length) return res.status(400).json({ errors: [...new Set(errors)] });
  req.userData = data;
  next();
};

// PUT: al menos un campo, y los que vengan deben ser válidos
exports.validateUpdate = (req, res, next) => {
  const data = pickAllowed(req.body);
  if (Object.keys(data).length === 0) {
    return res.status(400).json({ errors: ['Envía al menos uno de: ${ALLOWED_FIELDS.join(', ')}'] });
  }
  const errors = checkFields(data);

```

```
if (errors.length) return res.status(400).json({ errors });
req.userData = data;
next();
};

// :id debe ser un entero positivo
exports.validateId = (req, res, next) => {
  const id = Number(req.params.id);
  if (!Number.isInteger(id) || id <= 0) {
    return res.status(400).json({ errors: ['El id debe ser un número entero positivo'] });
  }
  req.userId = id;
  next();
};
```

 **Seguridad:** `pickAllowed()` es la defensa contra la asignación masiva: aunque el cliente envíe `{"name": "Ana", "email": "ana@mail.com", "role": "admin", "id": 999}`, al controlador solo llegan `name` y `email`.

Paso 8. Crear el controlador

controllers/userController.js

```
const User = require('../models/userModel');

// Express 5 envía automáticamente al manejador de errores
// cualquier excepción de una función async, así que no hace falta try/catch.

exports.getUsers = async (req, res) => {
  res.json(await User.getAll());
};

exports.getUser = async (req, res) => {
  const user = await User.getById(req.userId);
  if (!user) return res.status(404).json({ error: 'Usuario no encontrado' });
  res.json(user);
};

exports.createUser = async (req, res) => {
  const id = await User.create(req.userData);
  res.status(201).json(await User.getById(id));
};

exports.updateUser = async (req, res) => {
  const found = await User.update(req.userId, req.userData);
  if (!found) return res.status(404).json({ error: 'Usuario no encontrado' });
  res.json(await User.getById(req.userId));
};

exports.deleteUser = async (req, res) => {
  const found = await User.delete(req.userId);
  if (!found) return res.status(404).json({ error: 'Usuario no encontrado' });
  res.status(204).end();
};
```

El controlador no tiene **try/catch**: en Express 5 cualquier error de una función **async** llega automáticamente al manejador global que crearás en el Paso 10.

Paso 9. Definir las rutas

routes/users.js

```
const express = require('express');
const router = express.Router();
const usersController = require('../controllers/userController');
const { validateCreate, validateUpdate, validateId } = require('../middlewares/validation');

router.get('/', usersController.getUsers);
router.get('/:id', validateId, usersController.getUser);
router.post('/', validateCreate, usersController.createUser);
router.put('/:id', validateId, validateUpdate, usersController.updateUser);
router.delete('/:id', validateId, usersController.deleteUser);

module.exports = router;
```

Cada ruta encadena middlewares: por ejemplo, en **PUT** primero se valida el id, luego el cuerpo y recién después se llama al controlador. Si una validación falla, la cadena se corta y responde **400**.

Paso 10. Manejar los errores de forma centralizada

middlewares/errorHandler.js

```
// Rutas que no existen
exports.notFound = (req, res) => {
  res.status(404).json({ error: `Ruta no encontrada: ${req.method} ${req.originalUrl}` });
};

// Manejador global: registra el detalle en el servidor y responde sin exponerlo
exports.errorHandler = (err, req, res, next) => {
  if (err.type === 'entity.parse.failed') {
    return res.status(400).json({ error: 'El cuerpo de la petición no es JSON válido' });
  }
  if (err.code === 'ER_DUP_ENTRY') {
    return res.status(409).json({ error: 'Ya existe un usuario con ese email' });
  }
  console.error(err);
  res.status(500).json({ error: 'Error interno del servidor' });
};
```

 **Seguridad:** El detalle del error se registra en la consola del servidor con `console.error`, pero el cliente solo recibe un mensaje genérico. Enviar el error completo mostraría la consulta SQL y la estructura de tu base de datos.

Paso 11. Configurar la aplicación y el servidor

La configuración de Express (`app.js`) se separa del arranque (`index.js`). Así los tests pueden importar la app sin levantar el servidor ni conectarse a MySQL.

app.js

```

const express = require('express');
const cors = require('cors');
const userRoutes = require('./routes/users');
const { notFound, errorHandler } = require('./middlewares/errorHandler');

const app = express();

// CORS: si CORS_ORIGIN está definido (lista separada por comas) solo se aceptan esos orígenes
const origins = process.env.CORS_ORIGIN ? process.env.CORS_ORIGIN.split(',').map((o) => o.trim()) :
  '*';
app.use(cors({ origin: origins }));
app.use(express.json({ limit: '100kb' }));

app.use('/api/users', userRoutes);

app.use(notFound);
app.use(errorHandler);

module.exports = app;

```

index.js

```

require('./config');
const app = require('./app');
const db = require('./db');

const PORT = Number(process.env.PORT) || 3000;

async function start() {
  try {
    await db.query('SELECT 1');
    console.log('Conectado a MySQL');
  } catch (err) {
    console.error(`No se pudo conectar a MySQL (${err.code} || ${err.message}). Revisa las variables DB_*
    en .env`);
    process.exit(1);
  }

  app.listen(PORT, () => {
    console.log(`Servidor corriendo en http://localhost:${PORT}`);
  });
}

start();

```

Antes de escuchar peticiones, **index.js** ejecuta **SELECT 1** para comprobar la conexión. Si falla, muestra un mensaje claro y termina, en lugar de arrancar un servidor que no funcionará.

Paso 12. Ejecutar y probar manualmente

```
npm run dev
```

Debes ver **Conectado a MySQL** y **Servidor corriendo en http://localhost:3000**. El modo **dev** reinicia el servidor automáticamente cada vez que guardas un archivo. Luego, en Postman o Thunder Client, ejecuta estas pruebas **en orden** y completa la última columna:

#	Petición	Cuerpo (JSON)	Esperado	Obtenido
1	POST /api/users	{"name":"Ana","email":"ana@mail.com"}	201	

#	Petición	Cuerpo (JSON)	Esperado	Obtenido
2	POST /api/users	{"name":"Luis","email":"luis@mail.com","role":"admin","id":999}	201, sin role, id asignado por la BD	
3	POST /api/users	{"name":"Ana 2","email":"ana@mail.com"}	409	
4	POST /api/users	{}	400 con 2 errores	
5	GET /api/users	—	200, lista de 2	
6	GET /api/users/1	—	200	
7	GET /api/users/99	—	404	
8	GET /api/users/abc	—	400	
9	PUT /api/users/1	{"name":"Ana María"}	200	
10	PUT /api/users/1	{"foo":1}	400	
11	DELETE /api/users/2	—	204	
12	DELETE /api/users/2	—	404	
13	GET /api/productos	—	404	

 **Nota:** En Postman, para enviar JSON elige **Body › raw › JSON**. Si olvidas la cabecera **Content-Type: application/json**, el servidor no leerá el cuerpo y responderá 400.

 **Punto de control:** Las 13 pruebas devuelven el código esperado. Guarda capturas de las pruebas 2, 3, 7 y 11 para tu entrega.

Paso 13. Escribir pruebas automatizadas

Las pruebas manuales no escalan: cada cambio obligaría a repetir las 13. Node 20 incluye un ejecutor de pruebas (**node:test**) y **supertest** permite enviar peticiones a la app sin abrir un puerto. Para que los tests no dependan de MySQL, se **reemplazan los métodos del modelo** por versiones en memoria (dobles de prueba). Crea **test/users.test.js**:

```
test/users.test.js
// Tests de la API sin base de datos: se reemplazan los métodos del modelo por dobles en memoria.
const { test, beforeEach, after } = require('node:test');
const assert = require('node:assert/strict');
const request = require('supertest');

const User = require('../models/userModel');
const db = require('../db');
const app = require('../app');

let users;
beforeEach(() => {
  users = [{ id: 1, name: 'Ana', email: 'ana@mail.com' }];
  User.getAll = async () => users;
  User.getById = async (id) => users.find((u) => u.id === id) || null;
  User.create = async (data) => {
```

```

    const id = users.length + 1;
    users.push({ id, ...data });
    return id;
  };
  User.update = async (id, data) => {
    const u = users.find((x) => x.id === id);
    if (u) Object.assign(u, data);
    return Boolean(u);
  };
  User.delete = async (id) => {
    const before = users.length;
    users = users.filter((u) => u.id !== id);
    return users.length < before;
  };
});
after(() => db.end());

test('GET /api/users lista usuarios', async () => {
  const res = await request(app).get('/api/users').expect(200);
  assert.equal(res.body.length, 1);
});

test('GET /api/users/:id devuelve 404 si no existe', async () => {
  await request(app).get('/api/users/99').expect(404);
});

// TODO: escribe aquí tus propias pruebas (ver instrucciones)

```

Ejecuta **npm test**. Luego **agrega al menos cuatro pruebas propias** que verifiquen: (a) que un id no numérico devuelve 400; (b) que el POST ignora los campos **id** y **role**; (c) que un email duplicado devuelve 409 (simula el error con **code: 'ER_DUP_ENTRY'**); y (d) que un error interno responde 500 con el mensaje genérico, sin detalles de SQL.

✓ **Punto de control:** **npm test** muestra al menos 6 pruebas y 0 fallidas (# **fail 0**).

Paso 14. Versionar y publicar en GitHub

```

git init
git add .
git commit -m "API REST de usuarios con Node 20, Express 5 y MySQL"
git branch -M main
git remote add origin https://github.com/TU_USUARIO/api-node-2025.git
git push -u origin main

```

✓ **Punto de control:** En GitHub aparece tu proyecto **sin** la carpeta **node_modules** y **sin** el archivo **.env**.

5. Errores frecuentes y cómo resolverlos

Mensaje o síntoma	Causa probable	Solución
ECONNREFUSED	MySQL no está iniciado o el puerto no coincide	Inicia MySQL y revisa DB_PORT en .env (3306 o 3309)
ER_ACCESS_DENIED_ERROR	Usuario o contraseña incorrectos	Revisa DB_USER y DB_PASSWORD
ER_BAD_DB_ERROR	La base de datos no existe	Ejecuta schema.sql (Paso 2)
ER_NO_SUCH_TABLE	Falta la tabla users	Ejecuta schema.sql completo
EADDRINUSE: :::3000	Otro proceso usa el puerto 3000	Cierra el otro servidor o cambia PORT en .env

Mensaje o síntoma	Causa probable	Solución
process.loadEnvFile is not a function	Versión de Node menor a 20.12	Actualiza Node: nvm install 20 y nvm use 20
400 "El cuerpo de la petición no es JSON válido"	JSON mal escrito (comas, comillas)	Valida el JSON; usa comillas dobles
400 "name es obligatorio" aunque sí enviaste los datos	Falta la cabecera Content-Type: application/json	En Postman usa Body › raw › JSON

6. Lista de cotejo

Entregable: el enlace a tu repositorio de GitHub y un PDF de evidencias con las capturas del Paso 12, la salida de `npm test` y tus respuestas a las preguntas de reflexión. Nombre del PDF: `Apellido_Nombre_GuiaPractica.pdf`.

El docente verificará cada criterio. Antes de entregar, márcalo tú mismo en la columna **Autoevaluación**.

N.º	Criterio	Puntaje	Autoevaluación	Docente
1	La API arranca con <code>npm start</code> , se conecta a MySQL y lee la configuración desde <code>.env</code> .	2		
2	El código respeta la arquitectura por capas (rutas, controlador, modelo, middlewares).	3		
3	Todas las consultas SQL son parametrizadas y no usan <code>SELECT *</code> .	2		
4	La validación usa lista blanca: los campos no permitidos se ignoran.	2		
5	Los endpoints responden los códigos correctos (200, 201, 204, 400, 404, 409).	4		
6	Los errores internos se manejan en un único middleware y no exponen detalles de SQL.	2		
7	Las 13 pruebas manuales del Paso 12 tienen evidencia.	2		
8	<code>npm test</code> pasa con al menos 6 pruebas.	3		
	Total	20		

⚠ Importante: El repositorio no debe contener `node_modules` ni `.env`. Si contiene el archivo `.env` con credenciales, el criterio 1 se califica con 0.

7. Preguntas de reflexión

Responde brevemente, con tus propias palabras, al final de tu PDF de evidencias:

- ¿Qué habría pasado en la prueba 2 del Paso 12 si el modelo usara `INSERT INTO users SET ?` con `req.body` sin filtrar?
- ¿Por qué conviene separar `app.js` de `index.js`? ¿Qué ganaste con eso en el Paso 13?
- ¿Qué diferencia hay entre responder 400 y 404? Da un ejemplo de cada uno en esta API.
- ¿Por qué el controlador no necesita `try/catch` en Express 5? ¿Qué pasaría en Express 4?
- Si detienes MySQL con la API encendida, ¿qué responde un `GET /api/users`? ¿Y cuando MySQL vuelve? Pruébalo y explica el papel del pool.

8. Referencias

Express.js. (s. f.). *Error handling*. <https://expressjs.com/en/guide/error-handling.html>

Express.js. (s. f.). *Migrating to Express 5*. <https://expressjs.com/en/guide/migrating-5.html>

MDN Web Docs. (s. f.). *Códigos de estado de respuesta HTTP*. Mozilla.
<https://developer.mozilla.org/es/docs/Web/HTTP/Status>

MySQL2. (s. f.). *Quickstart*. <https://sidorares.github.io/node-mysql2/docs>

Node.js. (s. f.). *Process: process.loadEnvFile(path)*. En Node.js v20 documentation. <https://nodejs.org/docs/latest-v20.x/api/process.html>

Node.js. (s. f.). *Test runner*. En Node.js v20 documentation. <https://nodejs.org/docs/latest-v20.x/api/test.html>

OWASP Foundation. (s. f.). *Mass assignment cheat sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html